

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Автоматизированное проектирование цифровых устройств

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

на тему

«Использование модулей памяти»

Вариант 4

Выполнил: студент группы 250541

Власов Р. Е.

Проверил: Воронов А.А.

Минск 2026

1 Цель работы

Целью лабораторной работы является разработка и проверка VHDL-модели цифрового устройства, включающего постоянную память ROM, оперативную память RAM с общей шиной данных и управляющий автомат переноса слова из ячейки-источника в ячейку-приемник.

Дополнительно требуется подготовить проект в двух САПР: Intel Quartus II и Xilinx Vivado, получить временные диаграммы, RTL-представление, результаты компиляции и синтеза, а также подтвердить работоспособность автоматическим testbench.

2 Задание и исходные данные

По методическим указаниям необходимо разработать блок, в который входят модули памяти `lpm_rom` и `lpm_ram_io`, подключенные к общей восьмиразрядной шине. Требуется продемонстрировать чтение из памяти и выполнить пересылку данных из ROM в RAM через восьмиразрядный регистр.

Таблица 1 – Исходные данные

Параметр	Значение для варианта 4
Синхронность ввода <code>lpm_rom</code>	Асинхронный ввод адреса
Синхронность вывода <code>lpm_rom</code>	Синхронный вывод данных
Синхронность ввода <code>lpm_ram_io</code>	Синхронная запись данных
Синхронность вывода <code>lpm_ram_io</code>	Синхронное чтение данных
Ячейка-источник	ROM[4]
Ячейка-приемник	RAM[5]
Передаваемое значение	0x5A

3 Теоретические сведения

Постоянная память ROM хранит заранее заданный набор слов и в данной работе используется как источник данных. Оперативная память RAM допускает запись и последующее чтение по адресу. В FPGA внутренние двунаправленные шины и высокоимпедансные состояния обычно преобразуются синтезатором в логические мультиплексоры, однако в функциональной модели VHDL состояние 'Z' удобно показывает, какой узел в данный момент управляет общей шиной.

Синхронный вывод памяти означает, что новое значение появляется на выходе только после фронта тактового сигнала. Поэтому для варианта 4 чтение ROM и чтение RAM имеют задержку на один такт. Это учитывается управляющим автоматом: после включения ROM выполняется отдельное состояние загрузки регистра, а после записи в RAM выполняется состояние синхронного чтения.

Модель `lpm_rom` реализована как массив из шестнадцати восьмиразрядных слов. Модель `lpm_ram_io` содержит массив RAM 16 x 8, вход записи, вход разрешения вывода и общий порт `data_io`. При записи значение берется с общей шины, а при чтении RAM сама выводит слово на эту шину.

Иерархическая структура блока `memory_transfer`

Данные проходят по общей 8-разрядной шине: ROM[4] -> регистр -> RAM[5].

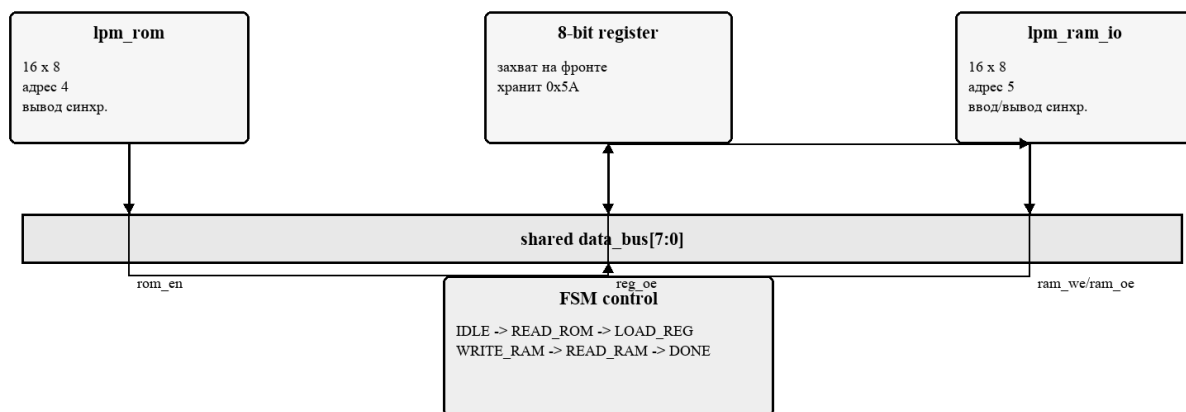


Рисунок 3 - Иерархическое представление разрабатываемого блока

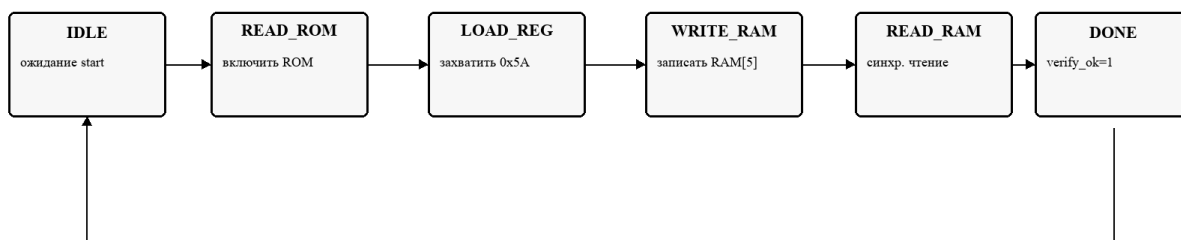


Рисунок 4 - Граф состояний управляющего автомата

4 Разработка VHDL-модели

Главный модуль `memory_transfer` содержит константы адресов источника и приемника, экземпляры ROM и RAM, регистр данных и конечный автомат. Вариант 4 задан константами `SRC_ADDR_C = 4`, `DST_ADDR_C = 5` и `EXPECTED_C = 0x5A`.

Автомат имеет шесть состояний: S_IDLE, S_READ_ROM, S_LOAD_REG, S_WRITE_RAM, S_READ_RAM и S_DONE. В S_READ_ROM включается ROM, в S_LOAD_REG регистр захватывает слово 0x5A, в S_WRITE_RAM регистр выдает слово на шину и активируется запись RAM, в S_READ_RAM выполняется синхронное чтение RAM, а в S_DONE формируются done_o и verify_ok_o.

Таблица 2 – Сигналы для разработки и моделирования модуля

Сигнал	Назначение
clk_i	Тактовый сигнал всех синхронных элементов.
rst_i	Сброс автомата в состояние ожидания и очистка регистра.
start_i	Запуск операции переноса.
data_bus_o	Наблюдаемая общая шина данных.
rom_q_o, reg_q_o, ram_q_o	Контрольные выходы ROM, регистра и RAM для моделирования.
done_o	Операция переноса завершена.
verify_ok_o	Прочитанное из RAM значение совпадает с ожидаемым 0x5A.

5 Функциональное моделирование

Testbench tb_memory_transfer формирует тактовый сигнал с периодом 10 ns, снимает reset, подает start и по тактам проверяет состояние автомата, адреса памяти, состояние общей шины, содержимое регистра и флаг verify_ok. Проверки выполнены операторами assert, поэтому ошибка сразу завершила бы моделирование с severity failure.

Параметр	Значение
Reset	FSM возвращается в IDLE, регистр очищается
ROM read	адрес источника равен 4
Sync ROM out	0x5A появляется после тактового фронта
Register load	reg_q захватывает 0x5A
RAM write	запись в адрес 5 при ram_we=1
Sync RAM read	данные RAM[5] читаются на следующем такте
Done	done=1 и verify_ok=1

Рисунок 5 - Покрывание проверок testbench

6 Реализация проекта в Intel Quartus II

Для Quartus подготовлены файлы `memory_transfer.qpf` и `memory_transfer.qsf`. Верхним уровнем назначен `memory_transfer_rtl_view_top`, в котором биты общей шины выведены отдельными портами `BUS_D0...BUS_D7` для удобного просмотра RTL-схемы.

В методических указаниях указано семейство Flex10K. Установленная версия Quartus II 9.1 Web Edition не принимает строки FAMILY "FLEX10K" и FAMILY "FLEX10KE", что зафиксировано в логе проверки. Для получения полной компиляции и отчетов проект собран на доступном семействе Stratix II. VHDL-описание при этом не изменялось.

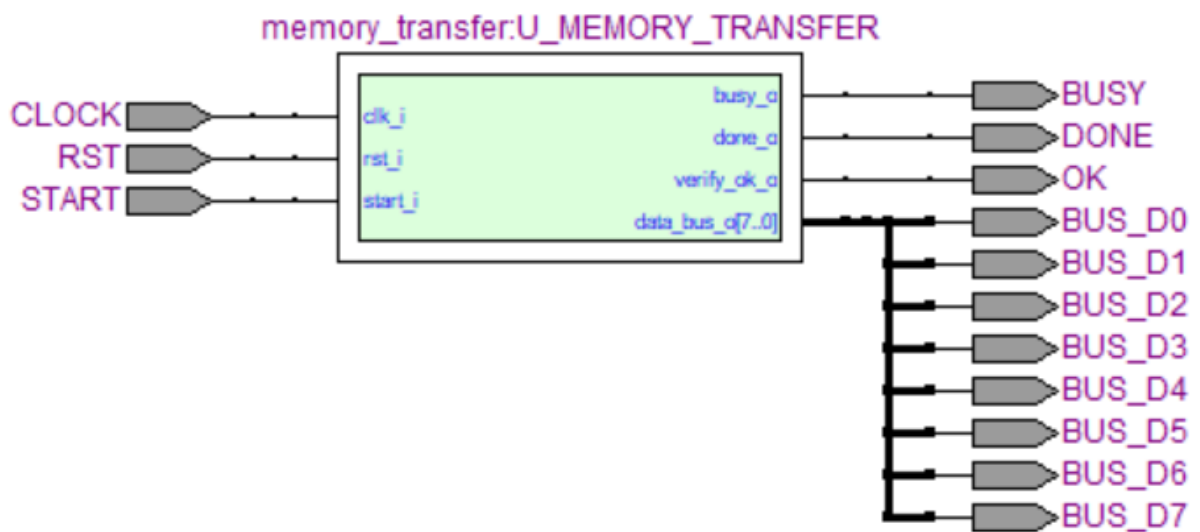


Рисунок 6 - RTL-представление проекта в Quartus II

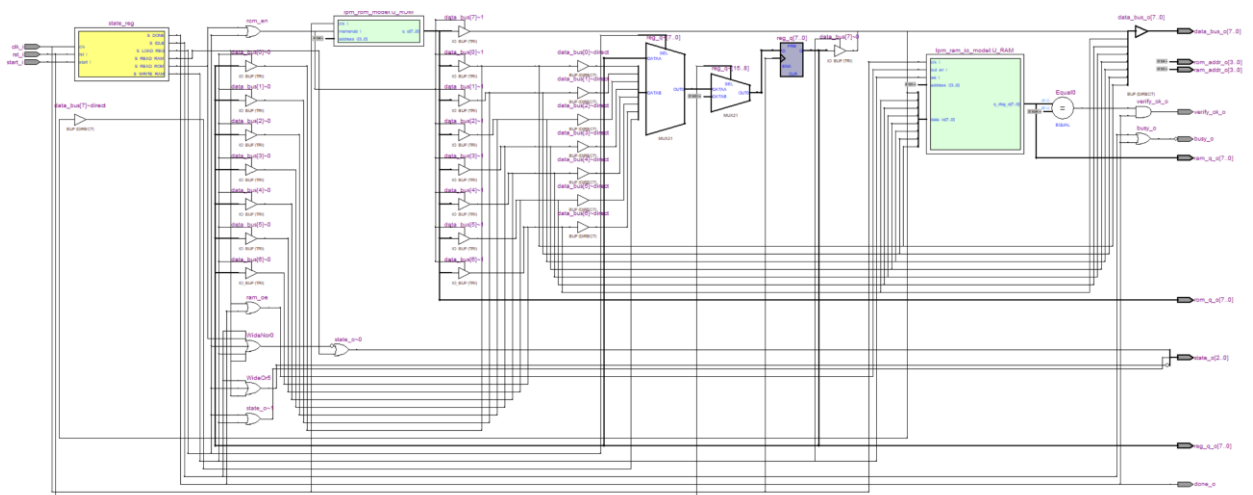


Рисунок 7 – Структурная схема проекта в Quartus

Quartus успешно выполнил Analysis & Synthesis, Fitter, Assembler и Classic Timing Analyzer. В отчете синтеза RAM распознана как altsyncram, что подтверждает аппаратную интерпретацию массива памяти, а не только поведенческое моделирование.

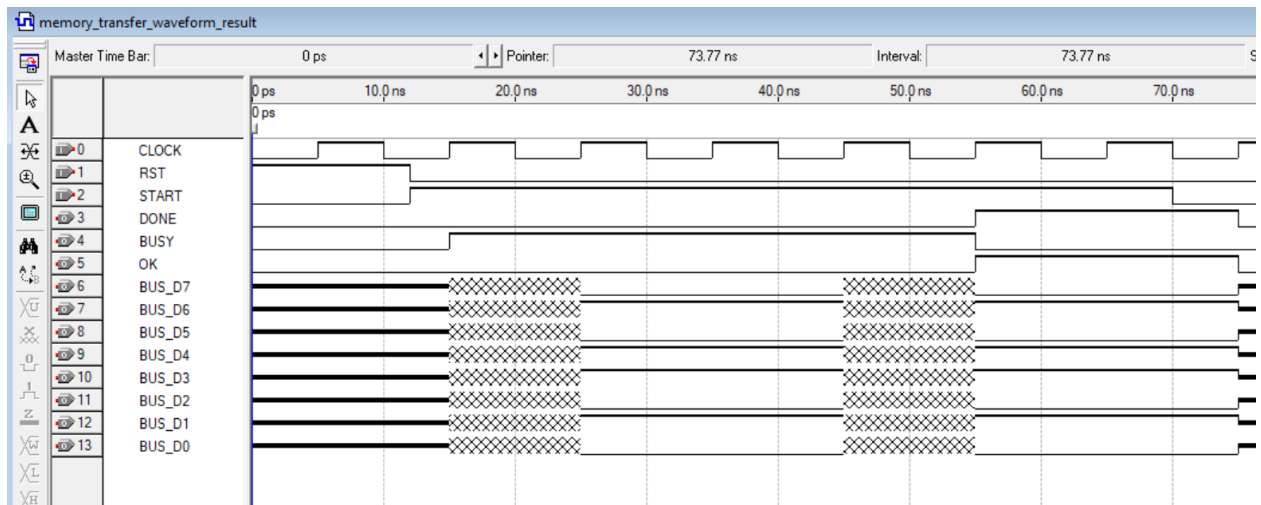


Рисунок 8 – Временная диаграмма проекта в Quartus

По временной диаграмме видно, что ROM начинает выдавать 0x5A после тактового фронта, затем это значение фиксируется в регистре, после чего регистр управляет общей шиной при записи в RAM[5]. На завершающем этапе RAM синхронно выводит 0x5A, и флаг verify_ok_o становится равным единице.

7 Реализация проекта в Xilinx Vivado

Для Vivado подготовлены create_project.tcl, memory_transfer.xdc и скрипт run_vivado_synth.ps1.

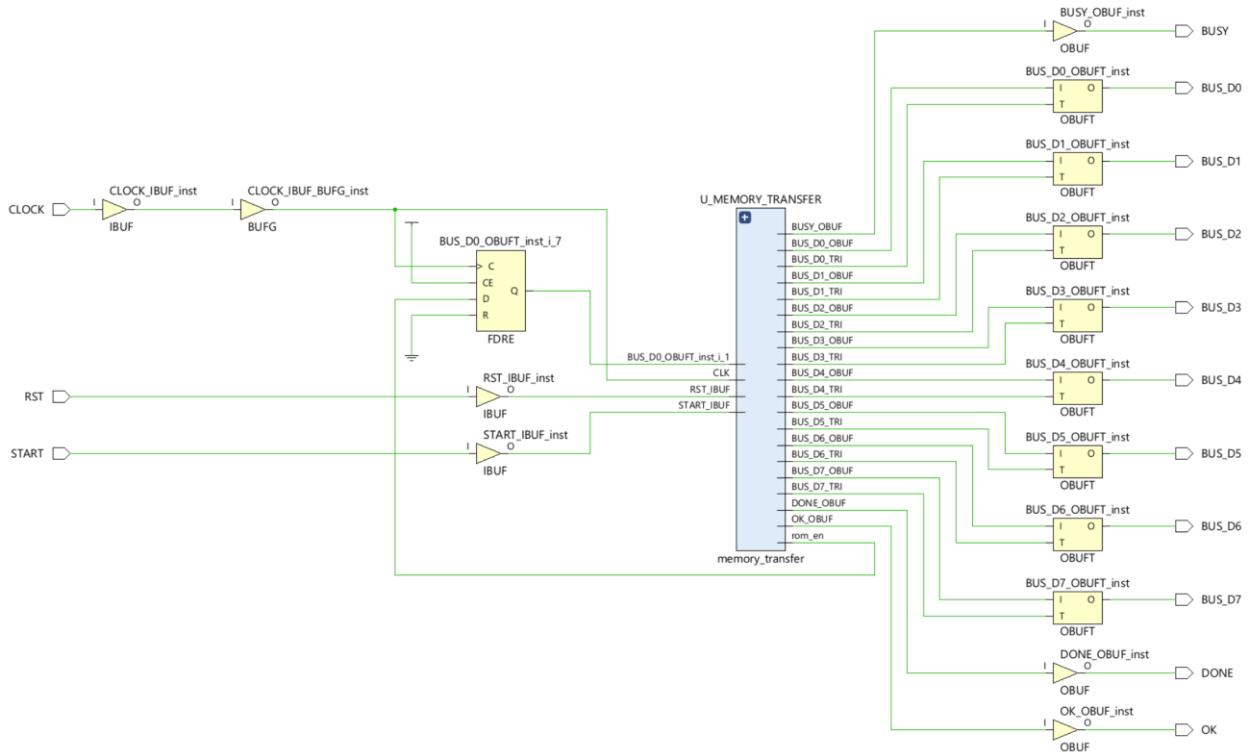


Рисунок 9 - RTL-представление проекта для Vivado

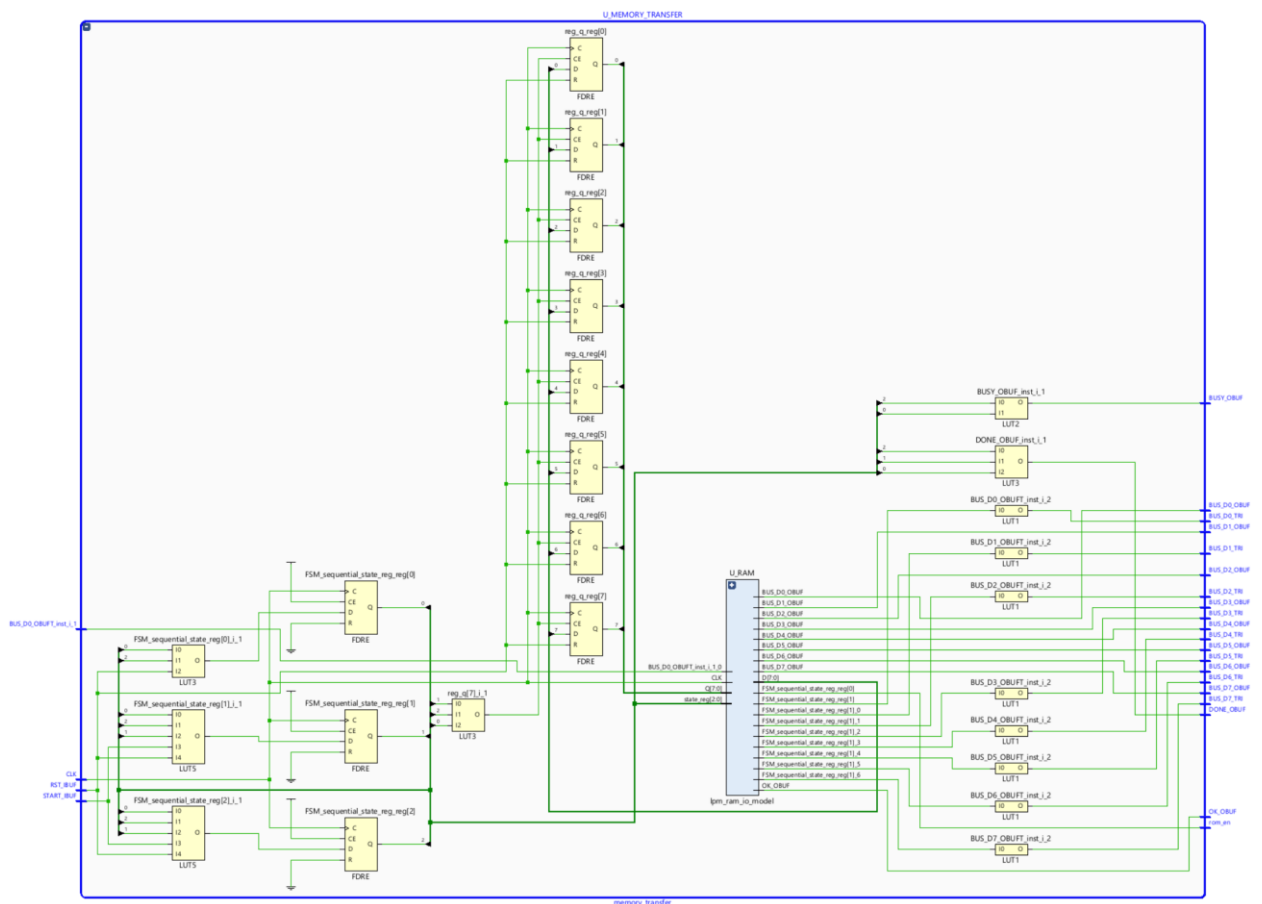


Рисунок 10 - RTL-представление внутри модуля u_memory_transfer

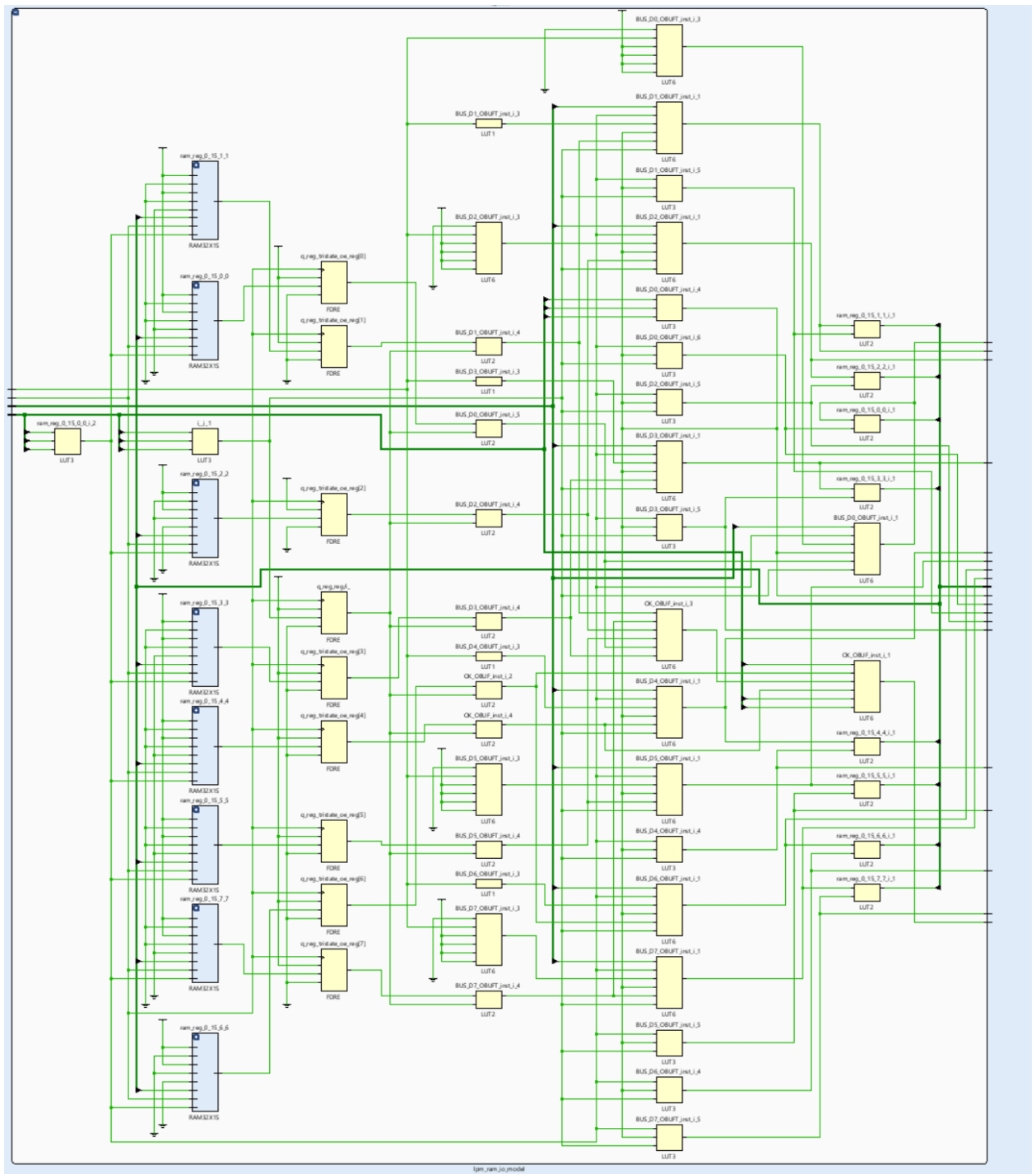


Рисунок 11 - RTL-представление внутри модуля u_ram



Рисунок 12 – Временная диаграмма проекта в Vivado

8 Вывод

В ходе лабораторной работы разработана VHDL-модель блока памяти по варианту 4. Реализованы модели ROM и RAM, общая шина данных, восьмиразрядный регистр и управляющий автомат переноса данных из ROM[4] в RAM[5].

Работоспособность подтверждена автоматическим testbench и временными диаграммами GHDL. Проект успешно собран в Quartus II и синтезирован в Xilinx Vivado. Получены RTL-представления, отчеты ресурсов и временные диаграммы.

Приложение А. Листинги VHDL-модулей

Модель ROM

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity lpm_rom_model is
  port (
    clk_i      : in  std_logic;
    address_i   : in  std_logic_vector(3 downto 0);
    memenab_i   : in  std_logic;
    q_o        : out std_logic_vector(7 downto 0)
  );
end entity;

architecture rtl of lpm_rom_model is
  type memory_t is array (0 to 15) of std_logic_vector(7 downto 0);
  signal q_reg : std_logic_vector(7 downto 0) := (others => 'Z');

  constant ROM_C : memory_t := (
    0 => x"11",
    1 => x"27",
    2 => x"3C",
    3 => x"45",
    4 => x"5A",
    5 => x"6E",
    6 => x"73",
    7 => x"8F",
    8 => x"91",
    9 => x"A4",
    10 => x"B8",
    11 => x"C2",
    12 => x"D5",
    13 => x"E9",
    14 => x"F0",
    15 => x"0D"
  );
begin
  process(clk_i)
  begin
    if rising_edge(clk_i) then
      if memenab_i = '1' then
        q_reg <= ROM_C(to_integer(unsigned(address_i)));
      else
        q_reg <= (others => 'Z');
      end if;
    end if;
  end process;

  q_o <= q_reg;
end architecture;
```

Модель RAM

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

```

entity lpm_ram_io_model is
  port (
    clk_i      : in      std_logic;
    address_i   : in      std_logic_vector(3 downto 0);
    we_i       : in      std_logic;
    out_en_i    : in      std_logic;
    data_io     : inout   std_logic_vector(7 downto 0);
    q_dbg_o     : out     std_logic_vector(7 downto 0)
  );
end entity;

architecture rtl of lpm_ram_io_model is
  type memory_t is array (0 to 15) of std_logic_vector(7 downto 0);
  signal ram : memory_t := (others => (others => '0'));
  signal q_reg : std_logic_vector(7 downto 0) := (others => 'Z');
begin
  process(clk_i)
  begin
    if rising_edge(clk_i) then
      if we_i = '1' then
        ram(to_integer(unsigned(address_i))) <= data_io;
      end if;

      if out_en_i = '1' and we_i = '0' then
        q_reg <= ram(to_integer(unsigned(address_i)));
      else
        q_reg <= (others => 'Z');
      end if;
    end if;
  end process;

  data_io <= q_reg when out_en_i = '1' and we_i = '0' else (others => 'Z');
  q_dbg_o <= q_reg;
end architecture;

```

Управляющий модуль memory_transfer

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity memory_transfer is
  port (
    clk_i      : in      std_logic;
    rst_i      : in      std_logic;
    start_i    : in      std_logic;
    done_o     : out     std_logic;
    busy_o     : out     std_logic;
    verify_ok_o : out     std_logic;
    state_o    : out     std_logic_vector(2 downto 0);
    data_bus_o : out     std_logic_vector(7 downto 0);
    rom_q_o    : out     std_logic_vector(7 downto 0);
    reg_q_o    : out     std_logic_vector(7 downto 0);
    ram_q_o    : out     std_logic_vector(7 downto 0);
    rom_addr_o : out     std_logic_vector(3 downto 0);
    ram_addr_o : out     std_logic_vector(3 downto 0)
  );
end entity;

architecture rtl of memory_transfer is
  constant SRC_ADDR_C : std_logic_vector(3 downto 0) := x"4";

```

```

constant DST_ADDR_C : std_logic_vector(3 downto 0) := x"5";
constant EXPECTED_C : std_logic_vector(7 downto 0) := x"5A";

type state_t is (S_IDLE, S_READ_ROM, S_LOAD_REG, S_WRITE_RAM, S_READ_RAM,
S_DONE);

signal state_reg : state_t := S_IDLE;
signal state_next : state_t;
signal data_bus : std_logic_vector(7 downto 0);
signal rom_q : std_logic_vector(7 downto 0);
signal ram_q : std_logic_vector(7 downto 0);
signal reg_q : std_logic_vector(7 downto 0) := (others => '0');

signal rom_en : std_logic;
signal reg_oe : std_logic;
signal ram_we : std_logic;
signal ram_oe : std_logic;
begin
  U_ROM : entity work.lpm_rom_model
    port map (
      clk_i      => clk_i,
      address_i  => SRC_ADDR_C,
      memenab_i  => rom_en,
      q_o        => rom_q
    );

  U_RAM : entity work.lpm_ram_io_model
    port map (
      clk_i      => clk_i,
      address_i  => DST_ADDR_C,
      we_i       => ram_we,
      out_en_i   => ram_oe,
      data_io    => data_bus,
      q_dbg_o    => ram_q
    );

  data_bus <= rom_q when rom_en = '1' else (others => 'Z');
  data_bus <= reg_q when reg_oe = '1' else (others => 'Z');

  process(state_reg, start_i)
  begin
    state_next <= state_reg;
    case state_reg is
      when S_IDLE =>
        if start_i = '1' then
          state_next <= S_READ_ROM;
        end if;
      when S_READ_ROM =>
        state_next <= S_LOAD_REG;
      when S_LOAD_REG =>
        state_next <= S_WRITE_RAM;
      when S_WRITE_RAM =>
        state_next <= S_READ_RAM;
      when S_READ_RAM =>
        state_next <= S_DONE;
      when S_DONE =>
        if start_i = '0' then
          state_next <= S_IDLE;
        end if;
    end case;
  end process;

  process(clk_i)

```

```

begin
  if rising_edge(clk_i) then
    if rst_i = '1' then
      state_reg <= S_IDLE;
      reg_q <= (others => '0');
    else
      if state_reg = S_LOAD_REG then
        reg_q <= data_bus;
      end if;
      state_reg <= state_next;
    end if;
  end if;
end process;

rom_en <= '1' when state_reg = S_READ_ROM or state_reg = S_LOAD_REG else
'0';
reg_oe <= '1' when state_reg = S_WRITE_RAM else '0';
ram_we <= '1' when state_reg = S_WRITE_RAM else '0';
ram_oe <= '1' when state_reg = S_READ_RAM or state_reg = S_DONE else '0';

done_o      <= '1' when state_reg = S_DONE else '0';
busy_o      <= '0' when state_reg = S_IDLE or state_reg = S_DONE else '1';
verify_ok_o <= '1' when state_reg = S_DONE and ram_q = EXPECTED_C else '0';

with state_reg select state_o <=
  "000" when S_IDLE,
  "001" when S_READ_ROM,
  "010" when S_LOAD_REG,
  "011" when S_WRITE_RAM,
  "100" when S_READ_RAM,
  "101" when others;

data_bus_o <= data_bus;
rom_q_o    <= rom_q;
reg_q_o    <= reg_q;
ram_q_o    <= ram_q;
rom_addr_o <= SRC_ADDR_C;
ram_addr_o <= DST_ADDR_C;
end architecture;

```

Верхний уровень для RTL Viewer

```

library ieee;
use ieee.std_logic_1164.all;

entity memory_transfer_rtl_view_top is
  port (
    CLOCK      : in  std_logic;
    RST        : in  std_logic;
    START      : in  std_logic;
    DONE       : out std_logic;
    BUSY       : out std_logic;
    OK         : out std_logic;
    BUS_D0     : out std_logic;
    BUS_D1     : out std_logic;
    BUS_D2     : out std_logic;
    BUS_D3     : out std_logic;
    BUS_D4     : out std_logic;
    BUS_D5     : out std_logic;

```

```

        BUS_D6      : out std_logic;
        BUS_D7      : out std_logic
    );
end entity;

architecture structural of memory_transfer_rtl_view_top is
    signal data_bus_s : std_logic_vector(7 downto 0);
    signal unused_state_s : std_logic_vector(2 downto 0);
    signal unused_rom_q_s : std_logic_vector(7 downto 0);
    signal unused_reg_q_s : std_logic_vector(7 downto 0);
    signal unused_ram_q_s : std_logic_vector(7 downto 0);
    signal unused_rom_addr_s : std_logic_vector(3 downto 0);
    signal unused_ram_addr_s : std_logic_vector(3 downto 0);
begin
    U_MEMORY_TRANSFER : entity work.memory_transfer
        port map (
            clk_i          => CLOCK,
            rst_i          => RST,
            start_i        => START,
            done_o          => DONE,
            busy_o          => BUSY,
            verify_ok_o    => OK,
            state_o         => unused_state_s,
            data_bus_o      => data_bus_s,
            rom_q_o         => unused_rom_q_s,
            reg_q_o         => unused_reg_q_s,
            ram_q_o         => unused_ram_q_s,
            rom_addr_o      => unused_rom_addr_s,
            ram_addr_o      => unused_ram_addr_s
        );

    BUS_D0 <= data_bus_s(0);
    BUS_D1 <= data_bus_s(1);
    BUS_D2 <= data_bus_s(2);
    BUS_D3 <= data_bus_s(3);
    BUS_D4 <= data_bus_s(4);
    BUS_D5 <= data_bus_s(5);
    BUS_D6 <= data_bus_s(6);
    BUS_D7 <= data_bus_s(7);
end architecture;

```

Приложение Б. Листинг testbench

tb_memory_transfer

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_memory_transfer is
end entity;

architecture sim of tb_memory_transfer is
    signal clk      : std_logic := '0';
    signal rst      : std_logic := '1';
    signal start    : std_logic := '0';
    signal done     : std_logic;
    signal busy     : std_logic;
    signal verify_ok : std_logic;
    signal state    : std_logic_vector(2 downto 0);
    signal data_bus : std_logic_vector(7 downto 0);
    signal rom_q    : std_logic_vector(7 downto 0);
    signal reg_q    : std_logic_vector(7 downto 0);
    signal ram_q    : std_logic_vector(7 downto 0);
    signal rom_addr : std_logic_vector(3 downto 0);
    signal ram_addr : std_logic_vector(3 downto 0);

    procedure tick(signal clk_s : in std_logic) is
    begin
        wait until rising_edge(clk_s);
        wait for 1 ns;
    end procedure;

begin
    clk <= not clk after 5 ns;

    DUT : entity work.memory_transfer
        port map (
            clk_i      => clk,
            rst_i      => rst,
            start_i    => start,
            done_o     => done,
            busy_o     => busy,
            verify_ok_o => verify_ok,
            state_o    => state,
            data_bus_o => data_bus,
            rom_q_o    => rom_q,
            reg_q_o    => reg_q,
            ram_q_o    => ram_q,
            rom_addr_o => rom_addr,
            ram_addr_o => ram_addr
        );

    stimulus : process
    begin
        tick(clk);
        rst <= '0';
        start <= '1';

        tick(clk);
        assert state = "001" report "FSM must enter ROM read state" severity
        failure;
        assert rom_addr = x"4" report "ROM source address must be 4" severity
        failure;
```

```

        tick(clk);
        assert state = "010" report "FSM must enter register load state" severity
failure;
        assert data_bus = x"5A" report "Register must see ROM[4] value 0x5A on
shared bus" severity failure;

        tick(clk);
        assert state = "011" report "FSM must enter RAM write state" severity
failure;
        assert reg_q = x"5A" report "Register must capture value 0x5A" severity
failure;
        assert ram_addr = x"5" report "RAM destination address must be 5"
severity failure;
        assert data_bus = x"5A" report "Register must drive value 0x5A during RAM
write" severity failure;

        tick(clk);
        assert state = "100" report "FSM must enter RAM readback state" severity
failure;

        tick(clk);
        assert done = '1' report "Transfer must finish with done=1" severity
failure;
        assert verify_ok = '1' report "Verification flag must be active after
successful transfer" severity failure;
        assert ram_q = x"5A" report "RAM[5] must contain transferred value 0x5A"
severity failure;
        assert data_bus = x"5A" report "RAM readback must drive 0x5A onto the
bus" severity failure;

        start <= '0';
        tick(clk);
        assert done = '0' report "FSM must return to idle after start is
released" severity failure;

        assert false report "tb_memory_transfer: TEST PASSED" severity note;
        wait;
    end process;
end architecture;

```


Приложение В. Файлы проектов САПР

Quartus QSF

```
set_global_assignment -name FAMILY "Stratix II"
set_global_assignment -name DEVICE AUTO
set_global_assignment -name TOP_LEVEL_ENTITY memory_transfer_rtl_view_top
set_global_assignment -name RESERVE_ALL_UNUSED_PINS "AS INPUT TRI-STATED"

set_global_assignment -name VHDL_FILE ../src/lpm_rom_model.vhd
set_global_assignment -name VHDL_FILE ../src/lpm_ram_io_model.vhd
set_global_assignment -name VHDL_FILE ../src/memory_transfer.vhd
set_global_assignment -name VHDL_FILE ../src/memory_transfer_rtl_view_top.vhd

set_global_assignment -name SIMULATION_MODE FUNCTIONAL
set_global_assignment -name VECTOR_OUTPUT_FORMAT VCD
set_global_assignment -name LAST_QUARTUS_VERSION 9.1

set_global_assignment -name PARTITION_NETLIST_TYPE SOURCE -section_id Top
set_global_assignment -name PARTITION_COLOR 16764057 -section_id Top
set_global_assignment -name LL_ROOT_REGION ON -section_id "Root Region"
set_global_assignment -name LL_MEMBER_STATE LOCKED -section_id "Root Region"
set_instance_assignment -name PARTITION_HIERARCHY root_partition -to | -
section_id Top
```

Vivado TCL

```
set script_dir [file dirname [file normalize [info script]]]
set root_dir [file normalize [file join $script_dir ".."]]
set project_dir [file join $script_dir "memory_transfer_vivado"]

create_project memory_transfer $project_dir -part xc7a35tcpg236-1 -force
set_property target_language VHDL [current_project]

add_files [file join $root_dir "src" "lpm_rom_model.vhd"]
add_files [file join $root_dir "src" "lpm_ram_io_model.vhd"]
add_files [file join $root_dir "src" "memory_transfer.vhd"]
add_files [file join $root_dir "src" "memory_transfer_rtl_view_top.vhd"]
add_files -fileset sim_1 [file join $root_dir "tb" "tb_memory_transfer.vhd"]
add_files -fileset constrs_1 [file join $script_dir "memory_transfer.xdc"]

set_property top memory_transfer_rtl_view_top [current_fileset]
set_property top tb_memory_transfer [get_filesets sim_1]

update_compile_order -fileset sources_1
update_compile_order -fileset sim_1

launch_runs synth_1
wait_on_run synth_1
open_run synth_1 -name synth_1
write_checkpoint -force [file join $project_dir "memory_transfer_synth.dcp"]
```